

Security Evaluation of AI-Generated Code Against Cross-Site Scripting (XSS) Vulnerabilities

Dan McCabe

Abstract

The rapid adoption of generative AI for code generation has started to raise some new security concerns, especially the prevalence of vulnerabilities in AI-generated code. This paper evaluates code generated by large language models (LLMs) and the susceptibility to cross-site scripting (XSS) attacks. Using a semi-automated testing pipeline, three freely available models (ChatGPT, Claude, and Gemini) were tested across eight web application prompts under standard and security-aware conditions. Static and dynamic analysis techniques were used to detect vulnerabilities and measure exploitability. Additionally, two different types of XSS payloads were tested: standard and obfuscated. Results show that all models produce XSS-vulnerable code when using standard prompting, with dynamic hit rates between 5.4% and 6.6%. Security-aware prompting was shown to dramatically reduce vulnerability, with Claude achieving a 0% dynamic hit rate. Obfuscated payloads unexpectedly performed worse than standard payloads, showing that the vulnerabilities don't come from inadequate filtering, but rather a complete lack of input sanitization. These findings highlight the need for improved awareness from developers and security-conscious prompting when using AI code generation tools.

Academic Integrity Pledge: I wrote this report myself, not using generative AI technologies.

1 Introduction

Within recent years, generative AI has rapidly transformed software development, allowing developers to create code with minimal effort. Tools like ChatGPT, Claude, and Gemini are now widely used for all facets of software engineering. However, this convenience comes with significant security risks that aren't entirely understood yet.

One major concern is their vulnerability to cross-site scripting attacks (XSS). XSS allows attackers to inject malicious scripts into web applications which could lead to session hijacking, data, theft, or unauthorized actions. Since many developers tend to trust and deploy this AI-generated code without any checks or reviews, these vulnerabilities can propagate directly into production systems.

This paper extends prior evaluations of baseline XSS vulnerability rates in AI-generated code by introducing two new dimensions: (1) security-aware defense prompts that instruct models to sanitize input and prevent XSS, and (2) obfuscated XSS payloads designed to bypass sanitization filters. The goal is to assess how vulnerable AI-generated code is to these attacks and if simple prompt engineering can serve as a practical mitigation.

2 Related Work

Prior work has established that AI-generated code frequently contains security vulnerabilities. Pearce et al. [1] showed that GitHub Copilot produces insecure code in approximately 40% of cases across

a variety of scenarios. Similarly, Khoury et al. [2] found that ChatGPT generates code that is vulnerable to SQL injection, buffer overflows, and XSS when prompts do not explicitly request secure implementations.

Research into prompt engineering for security has shown that explicitly instructing LLMs to follow secure coding practices improves the output quality [3]. However, the amount of improvement and the consistency across multiple models has not been thoroughly benchmarked with dynamic testing.

XSS remains a prevalent web vulnerability, consistently showing up in OWASP Top 10 reports. Obfuscated XSS techniques, including encoding bypasses, double URL encoding, and SVG/MathML injection, are commonly used to evade web application firewalls (WAFs) in real-world attacks [4]. Whether these techniques are necessary against AI-generated code has not been previously studied.

3 Threat Model

This study assumes the following threat scenario:

- A developer uses a free AI tool to create a single-file Flask application that has user input fields.
- The developer then deploys that code without review.
- An attacker uses XSS payloads crafted to attempt to execute JavaScript, steal cookies, or inject malicious content onto the site.

This threat model is a realistic and increasingly common pattern for deployment, particularly among developers who treat AI-generated code as production-ready.

4 Methodology

4.1 Overview

A semi-automated pipeline was created to evaluate the AI-generated code which involved 4 steps. Those steps were: (1) generate code using the LLMs using either their respective APIs or online chat tools, (2) deploy the applications in isolated Docker containers, (3) run static and dynamic security analysis, and (4) log the results to a database for analysis.

4.2 Implementation Constraints

To ensure that the generated code was consistent and easy to deploy and analyze, all prompts were told to be single-file Flask applications written in Python. This was done by giving the LLM explicit instructions to do so. This helped to eliminate any deployment and analysis issues that come with multi-file projects and allowed the pipeline to easily and uniformly deploy and analyze the outputs.

4.3 Models Evaluated

Three freely available large language models were evaluated: ChatGPT (GPT-5 mini), Claude (Sonnet 4.6), and Gemini (3.0 Flash). All

models were accessed through their respective online interfaces to reflect the typical conditions of individual developers.

4.4 Prompt Design

Eight prompts were created to simulate common web application features that involve user input: user profile display, login error handling, contact form, feedback form, comment section, search form, to-do list, and URL-based greeting. Each prompt was executed 10 times per model to reduce the variance between the outputs.

For each prompt, two variants were used:

- **Standard prompts:** Normal descriptions with no security requirements.
- **Defense prompts:** The same descriptions with instructions to sanitize all user inputs and prevent XSS vulnerabilities.

4.5 Static Analysis

Static analysis was performed on every generated code sample to detect common patterns associated with XSS vulnerabilities, including unsafe DOM manipulation (e.g., innerHTML assignment), lack of input sanitization, and use of dangerous functions that directly render user input. Each finding was then classified by severity, either Critical, High, Medium, or Low.

4.6 Dynamic Analysis

Dynamic testing involved deploying each generated application in a Docker container and injecting a set of XSS payloads. Two different payload categories were tested:

- **Standard payloads:** Basic XSS attack vectors including reflected script injection, basic script tags, event handlers, attribute injection, JavaScript URIs, and DOM XSS patterns.
- **Obfuscated payloads:** Encoding and evasion techniques including double URL encoding, HTML entity obfuscation, Unicode JavaScript escapes, base64 data URIs, SVG/MathML injection, fromCharCode encoding, template literals, mutation XSS, comment splitting, and WAF case/encoding combos.

4.7 Static-Dynamic Cross-Validation

To assess the reliability of the analysis methods, results from static and dynamic analysis were cross-validated per-sample. Each code sample was classified into one of three categories: True Positive (flagged by static and dynamic analysis), False Positive Candidate (flagged statically but not dynamically), and False Negative Candidate (dynamically exploited but not flagged statically). This cross-validation was performed across all six tool-and-prompt conditions to identify any systematic discrepancies between the two analysis methods and to quantify how often each method over- or under-reported vulnerabilities. True negatives were excluded from the logging to focus on confirmed vulnerabilities and disagreements rather than logging safe samples.

A *hit* was recorded when a payload successfully executed in the browser of the deployed application. Dynamic hit rate is the proportion of payload injection attempts that resulted in successful execution.

5 Results

5.1 Effect of Defense Prompts

The results showed that security-aware prompting had a dramatic effect on the dynamic exploitability. Claude with defense prompts achieved a 0.0% dynamic hit rate— a complete elimination of all exploitable vulnerabilities. ChatGPT reduced from 5.5% to 4.3%, and Gemini from 5.4% to 3.3% (Figure 1). Claude was the only model to completely eliminate dynamic exploitation under the security-aware prompting.

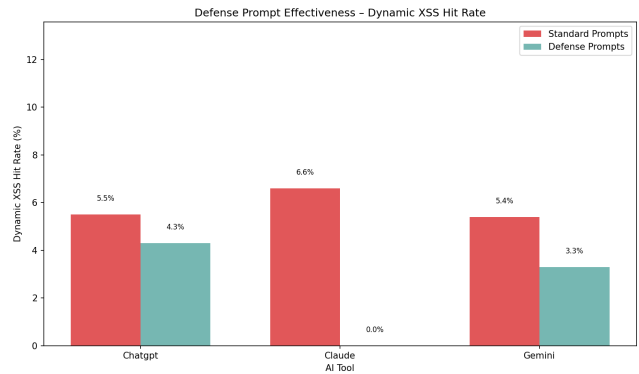


Figure 1: Defense prompt effectiveness on dynamic XSS hit rate.

5.2 Severity Distribution

The overwhelming majority of static findings across all of the AI tools and prompts were classified as Critical (Figure 2). Claude under normal prompting produced approximately 725 Critical findings, far exceeding all other tools (ChatGPT ~405, Gemini ~360). Under the security-aware prompting, Claude’s findings dropped dramatically to around 20, while ChatGPT and Gemini defensive code remained around the initial range. This suggests that Claude’s compliance with the defensive prompts is much more comprehensive than the other tools.

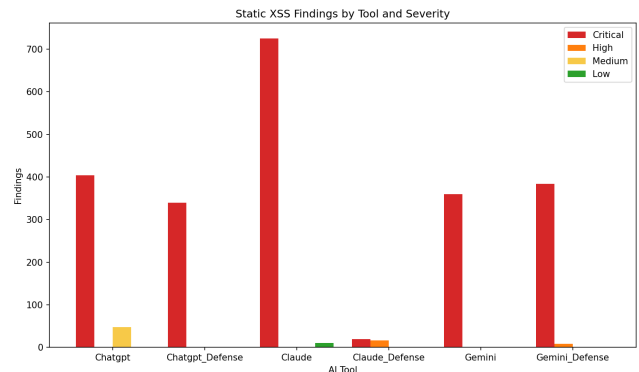


Figure 2: Static XSS findings by tool and severity.

5.3 Four-Way Combination Analysis

To fully characterize the interaction between prompt type and payload type, results were analyzed across all four combinations of standard/obfuscated payloads and standard/defense prompts per model (Figure 3).

Under the normal prompts, all three models exhibited similar hit rates with standard payloads: ChatGPT at 7.6%, Claude at 7.4%, and Gemini at 7.5%. Obfuscated payloads under standard prompting were consistently lower across all three models: 3.6% (ChatGPT), 6.0% (Claude), and 3.4% (Gemini).

Defensive prompting had noticeably different results depending on the model. Claude achieved a 0.0% hit rate under the defensive prompting for both standard and obfuscated payloads. ChatGPT and Gemini showed reductions, but not at the same level as Claude: ChatGPT fell from 7.6% to 6.4% with standard payloads and from 3.6% to 2.3% with obfuscated payloads; Gemini fell from 7.5% to 5.3% and from 3.4% to 1.6% respectively.

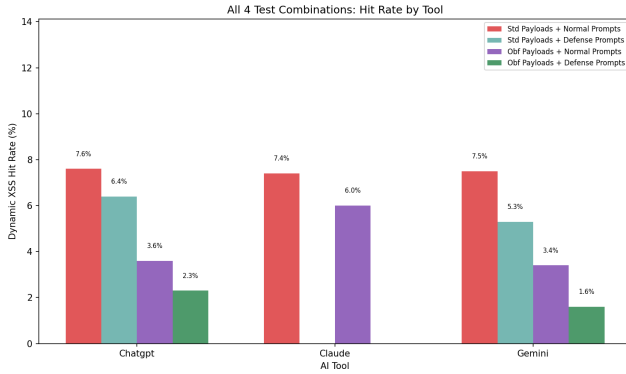


Figure 3: Dynamic XSS hit rate across all four combinations of prompt type and payload type per model.

5.4 Static-Dynamic Cross-Validation

To assess the agreement between static and dynamic analysis methods, each sample was classified into the three categories explained before (Figure 4):

- **True Positive (TP):** Both static and dynamic analysis agree the sample is vulnerable.
- **False Positive Candidate (FP):** Flagged by static analysis only; not dynamically exploitable.
- **False Negative Candidate (FN):** Exploited dynamically but not flagged statically.

True positives were the dominant category across all prompting conditions, confirming strong agreement between the two analysis methods. Claude under defense was the only notable exception, as there is a small sample size and all of the flags were static with no dynamic exploits found, consistent with its 0.0% dynamic hit rate.

False Negative Candidates were present across all standard-prompting tools, as well as ChatGPT under defensive prompting. These False Negative Candidates were most prominent in the defense variant of ChatGPT, suggesting the static analysis underreported exploitability in those conditions. False Positive Candidates were present across every condition, showing that there may be patterns of unsafe code in these samples, but they were not exploitable in this scenario.

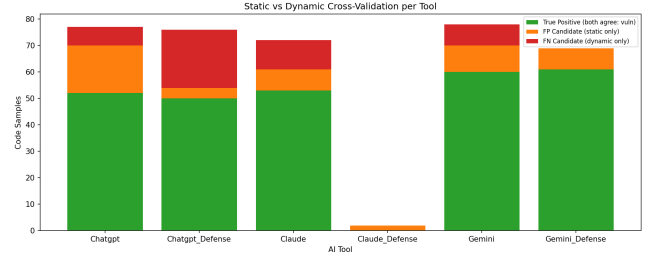


Figure 4: Static vs. dynamic cross-validation per tool. Green = True Positive (both agree: vulnerable); orange = FP Candidate (static only); red = FN Candidate (dynamic only).

5.5 Standard vs. Obfuscated Payload Performance

Standard payloads consistently outperformed obfuscated payloads in hit rate across all models (Figure 5). ChatGPT showed 7.6% standard payload hit rate versus 3.6% obfuscated; Claude showed 7.4% versus 6.0%; Gemini showed 7.5% versus 3.4%. The defensive prompting variant of Claude’s generated code achieved 0.0% hit rate for both payload types, so the numbers for Claude all came from the standard prompting variants.

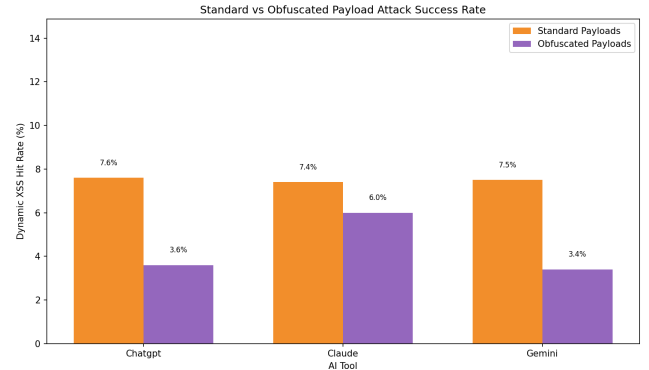


Figure 5: Standard vs. obfuscated payload attack success rate.

6 Discussion

6.1 Security is Not Implicit in AI Code Generation

All three models were shown to produce XSS-vulnerable code under standard prompting conditions. This confirms that LLMs focus more on function rather than security when generating code, which is consistent with previous research. Developers who treat this AI-generated code as production-ready without any code or security review are potentially exposing their applications to real, dangerous vulnerabilities.

6.2 Defense Prompts Are a Simple and Effective Mitigation

The most significant finding in this study is that explicitly instructing models to prevent XSS-vulnerabilities dramatically improves the security of the generated code. Claude’s complete elimination of dynamic hits under defense shows that these LLMs are capable of producing secure code, but doesn’t do so unless specifically asked. This could be to simplify the generated code on simple prompts, not to infer what the user wants, or other reasons, but adding a security clause to the prompts shows that it is able to yield substantial protection. The less dramatic effectiveness seen in ChatGPT and Gemini suggests that the compliance varies by model, and that the prompt formulation may benefit from further refinement.

The four-way combination results help reinforce and clarify the findings considerably. Claude’s 0.0% hit rate under defense prompting held for both payload categories, showing that when asked, it can create robust secure code. In contrast, ChatGPT and Gemini still showed vulnerabilities under defense prompting, suggesting that sanitization or filtering might be applied inconsistently. The fact that obfuscated payloads continued to under-perform standard payloads supports that fact that much of the code lacks sanitization entirely, so obfuscation gives no advantage to the attacker.

6.3 The Obfuscation Paradox

The finding that obfuscated payloads under-perform standard payloads is counterintuitive but can be clearly explained. Obfuscation techniques are meant to bypass sanitization filters and WAFs, however much of the AI-generated code contained no sanitization whatsoever. A basic script injection succeeded immediately because there was nothing to bypass. Obfuscated payloads, which rely on specific decoding behaviors and parsing quirks, encounter the absence of defenses but require those conditions to execute, resulting in a lower success rate. This implies that these vulnerabilities are fundamental and that simple input sanitization would increase security and force attackers to resort to more sophisticated techniques.

6.4 Cross-Validation Reveals Method-Specific Blind Spots

The cross-validation results show that neither static nor dynamic analysis are sufficient on their own. False Negative and False Positive Candidates were present across almost all situations, with the exception of False Negatives in the Claude and Gemini defensive prompts. This indicates that both techniques will flag some non-vulnerabilities and miss some others and that both are needed to come up with a more in-depth security picture. The consistency of False Positives across all situations shows that the static pattern matching can sometimes flag code that is not practically exploitable and tends to over-report vulnerabilities. Claude’s defense variant was the extreme case out of the six, with all of its findings being False Positive candidates, with the dynamic testing showing zero successful exploits. Overall, the majority of the findings were confirmed by both dynamic and static analyses, but the combination of both methods is able to provide a more accurate and complete picture of security posture rather than just one.

7 Limitations

- The study is limited to three freely accessible models; results may not generalize to paid or enterprise variants.
- Eight prompt types represent a small subset of real-world web application patterns.
- Static analysis relies on pattern matching, which may produce false positives and may miss vulnerability patterns.
- The Docker testing environment may not fully replicate all browser-side XSS execution contexts, potentially affecting dynamic hit rates.
- The generated code being a single-file Flask application in Python constrains the generalizability of the findings, as real-world applications typically span multiple files, with templating engines that have auto-escaping enabled or employ frontend frameworks that provide built-in XSS protections. Single-file Flask applications lack these safeguards, which could inflate vulnerability rates relative to more mature project configurations.

8 Conclusion

This study demonstrates that all three evaluated AI code generation tools produce XSS-vulnerable code under standard prompting conditions, with dynamic hit rates ranging from 5.4% to 6.6% and a majority of the static findings being classified as Critical severity. Security-aware prompting was able to substantially reduce that vulnerability, with Claude completely eliminating dynamically exploitable XSS. Obfuscated payloads were shown to under-perform standard payloads across all models, revealing that many vulnerabilities stem from a complete lack of input sanitization rather than ineffective filtering.

The implication for developers is clear: AI-generated code should not be deployed without security review, and security requirements should be explicitly stated in prompts. Future work will be able to extend this evaluation to additional popular models including GitHub Copilot and Microsoft Copilot, explore more prompts and payloads, explore more refined defense prompts, and investigate whether findings generalize to other vulnerabilities like SQL injection and CSRF.

References

- [1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in *Proc. IEEE Symposium on Security and Privacy*, 2022.
- [2] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, “How Secure is Code Generated by ChatGPT?” in *Proc. IEEE International Conference on Systems, Man, and Cybernetics*, 2023.
- [3] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?” in *Proc. ACM CCS*, 2023.
- [4] OWASP, “Cross Site Scripting Prevention Cheat Sheet,” OWASP Foundation, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html